

BUS MASTER:

The device in a computer that drives the address bus, and the bus control signals is called the bus master. In a simple computer architecture, only the CPU can be bus master, which means that all communications between I/O devices must involve the CPU. More complex architectures allow other devices (or multiple CPUs) to take turns at controlling the bus.

Although processors operate at high speeds, they are not always efficient at transferring data. When a processor transfers data from one device to another, it must use a bus cycle to read in the data from the source device and use another bus cycle to write the data back out to the destination device. The overhead in reading data into the processor and writing it out again greatly slows data transfers. The bus request operation allows other bus masters to take control of the system buses and rapidly transfer data between system devices.

Bus request operations are similar to interrupts and exceptions but differ in three important ways. Bus request operations do not complete the current instruction cycle before proceeding. Instructions can take hundreds or even thousands of clock cycles, and the circumstances that generated the bus request may be too urgent to be delayed.

Bus Contention:

Bus contention occurs when two or more devices try to output opposite logic levels on the same common bus line. The most common form of bus contention is when one device has not completely turned off before another device connected to the bus line is turned on. This generally occurs in memory systems when switching from the READ mode to the WRITE mode or vice versa and is the result of a timing problem.

Shared Signal Lines:

In computer architecture, **shared signal lines** refer to the use of a single communication line to transmit multiple signals. Shared signal lines are commonly used in computer systems to reduce the number of wires required for communication between devices. However, shared signal lines can cause **bus contention** when two or more devices try to output opposite logic levels on the same common bus line. This can occur when one device has not completely turned off before another device connected to the bus line is turned on. This generally occurs in memory systems when switching from the READ mode to the WRITE mode or vice versa and is the result of a timing problem

This technique is often used to:

- Save space on printed circuit boards (PCBs): By sharing wires, designers can reduce the overall footprint of the circuit, making it smaller and more compact.
- Reduce the number of wires needed: This can simplify the manufacturing process and make the final product more reliable.

Tri-State Output

Tri-state, or three-state, is a term in digital electronics that describes a special kind of digital output. It has three possible states: logic 0, logic 1, and high-impedance (high Z). High impedance means the output seems like it's "disconnected" from the electronic pathway.

This three-state feature is useful when connecting many outputs to a shared pathway. For instance, in computer buses, latches are used for buffering. They help control the flow of data, allowing two subsystems to share the same data bus.

When one subsystem uses the bus, the other sets its output state to high impedance, preventing any unwanted electric currents. A dedicated control line is typically used to enable or disable the tri-state function.

Fan-Out

The fan-out of a logic gate is the maximum number of inputs of the same series in an IC family that can be connected to a gate's output and still maintain the output voltage levels within specified limits. Fan-out is a significant parameter only for bipolar logic because of the type of circuit technology. Since very high impedances are associated with CMOS circuits, the fan-out is very high but depends on frequency because of capacitive effects. Fan-out is specified in terms of unit loads.

Buffers

A buffer is a specialized circuit designed to isolate the output of a device from the loading effects of other devices. Buffers play a crucial role in maintaining the integrity and reliability of digital signals in a circuit.

Function of Buffers:

- Buffers prevent the exceeding of device fan-outs by isolating the output from the loading effects of multiple input devices.
- They ensure that the performance specifications of the output device are maintained, particularly regarding voltages and timing.

Types of Buffers:

- Noninverting Buffers: These buffers maintain an identical output signal to the input signal.
- Other Types of Buffers: Various types include tri-state buffers (can be in a high-impedance state), inverting buffers (invert the input signal), bidirectional buffers (allow information flow in both directions on a data bus), and Schmitt triggers (prevent erratic behavior due to system noise affecting slowly changing inputs).

Device selection:

Device selection is a crucial aspect in the interaction between a processor and various system devices, such as ROM, RAM, and hardware I/O ports. The address bus of the processor is utilized to access these devices. To facilitate this process and ensure that the processor communicates with the intended device, each device is equipped with a special input known as chip select (CS) or chip enable (CE).

Essentially, when the processor needs to access a specific device, it asserts the select line of that particular device. This select line, or chip select, serves as a signal to enable the intended device and let it know that the processor is attempting to communicate with it.

In practical terms, providing separate control lines for each system device is not feasible in general-purpose computers due to the uncertainty of the devices a system may contain. Instead, designers employ programmable logic devices (PLDs) or dedicated hardware decoders. These components decode processor addresses and generate the necessary device select lines, streamlining the process of device selection in a computer system.

System Timing:

System timing in a computer refers to the coordination and synchronization of signals between the processor and other components like peripherals, memory, and devices. It's crucial to ensure that data is correctly stored and accessed.

One challenge in computer systems is that processors may run much faster than some peripherals or devices. This speed difference can lead to issues in how signals are handled. To address this, different types of system buses are used:

- **Synchronous Buses:** These use a synchronized clock to make sure that signals from the processor align properly with the setup and hold times of peripherals. Synchronous buses are generally faster.
- **Asynchronous Buses:** These automatically insert wait states during a bus cycle until a signal indicates that the cycle can finish. Wait states temporarily pause the read or write operation, allowing slower devices to catch up with the faster processor.
- **Semisynchronous Buses:** Similar to asynchronous buses, but they will complete the bus cycle unless a signal indicates otherwise. If needed, wait states can be inserted.

To manage the potential delays, techniques like CPU caches, instruction pipelines, prefetching, and simultaneous multithreading are used to optimize performance and minimize the impact of wait states, which can slow down the computer. Additionally, wait states are controlled by a logic circuit called a wait-state generator, which generates signals to indicate when the processor should pause during certain operations.

MicroProcessor:

A microprocessor is a central component within a computer system that functions as the brain of the machine, responsible for executing instructions and managing data processing tasks. It is a programmable integrated circuit (IC) that encompasses the functionalities of a computer's central processing unit (CPU). Let's delve into a more detailed explanation suitable for an exam context.

A microprocessor serves as the nucleus of a computer architecture. Its primary role revolves around interpreting and executing instructions from a computer program, which is typically loaded into the system's memory.

At its core, a microprocessor is composed of an arithmetic logic unit (ALU), which performs mathematical and logical operations, and a control unit that manages the flow of data within the processor and between other components of the computer system. Additionally, it encompasses registers, which act as high-speed storage units for temporary data and operands, enhancing the efficiency of data processing operations.

ALU:

The Arithmetic Logic Unit (ALU) is a fundamental component within a processor that plays a pivotal role in executing arithmetic and logic operations. Its primary function is to perform mathematical computations, such as addition, subtraction, multiplication, and division, as well as logical operations like AND, OR, and NOT. Let's delve into a more detailed explanation based on the provided text.

The ALU operates on data that is transferred into it from two key sources: the accumulator and the data register. Both the accumulator and the data register are 8-bit registers, capable of holding one byte of data each. The byte of data transferred into the ALU is referred to as an operand, as it undergoes manipulation by the ALU during arithmetic or logic operations.

Whenever the ALU performs an operation involving two operands, the outcome is consistently directed into the accumulator, effectively replacing the initial operand. This characteristic ensures that the ALU operates seamlessly within the processor, providing the capability to execute a diverse range of arithmetic and logic instructions.

Instruction Decoder:

Function: The instruction decoder is a crucial component of the processor responsible for translating the binary instruction code (op-code) received from memory into signals that the processor can use to perform specific operations.

Operation: When an instruction is fetched from memory and placed on the data bus, the instruction decoder interprets the op-code. This op-code specifies the operation the processor needs to perform, such as addition, subtraction, or data transfer.

Output: Once the op-code is decoded, the instruction decoder provides information to the timing and control unit regarding the specific operation to be executed.

Timing Unit:

Function: The timing unit, also known as the clocking unit, manages the synchronization and timing of various operations within the processor. It ensures that different parts of the processor work together in a coordinated manner.

Operation: Upon receiving information from the instruction decoder, the timing unit generates signals and controls the timing sequence for the execution of the instruction. It orchestrates the flow of data and operations within the processor to ensure accurate and synchronized execution.

Output: The timing unit produces signals that coordinate the execution of the instruction, ensuring that each step occurs at the appropriate time.

Control Unit:

Function: The control unit is responsible for directing and coordinating the activities of the various components within the processor. It interprets the signals from the instruction decoder and timing unit to generate control signals for other parts of the processor.

Operation: After receiving information from the instruction decoder and timing unit, the control unit generates control signals that manage the flow of data, direct operations, and control the functioning of registers and other components within the processor.

Output: The control unit's output includes signals that guide the execution of the instruction, ensuring that the processor performs the required operation accurately and efficiently.

Register set:

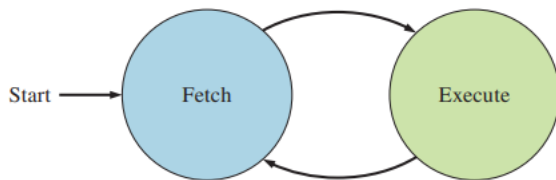
Processors typically have two categories of registers for temporary storage of data: general-purpose registers and special-purpose registers. General-purpose registers are used to store any type of data that may be required by a program. Special-purpose registers are dedicated to a specific function. Some typical special-purpose registers are described as follows.

1. Flag register: This register is sometimes called a condition code register or status register. It indicates the status of the contents of the accumulator or certain other conditions within the processor. For example, it can indicate a zero result, a negative result, the occurrence of a carry, or the occurrence of an overflow from the accumulator.

2. Program counter: This counter produces the sequence of memory addresses from which the program instructions are taken. The content of the program counter is always the memory address from which the next byte is to be taken. In some processors, the program counter is known as the instruction pointer.
3. Address register: This register temporarily stores an address from the program counter in order to place it on the address bus. As soon as the program counter loads an address into the address register, it is incremented (increased by 1) to the address of the next instruction.
4. Stack pointer: The stack pointer is a register that is mainly used during program subroutines and interrupts. It is used in conjunction with the memory stack.
5. Index register: The index register is used as one means of addressing the memory in a mode of addressing called indexed addressing.

Instruction Execution Cycle:

The instruction execution cycle, commonly known as the Fetch/Execute Cycle, is a fundamental process that a processor undergoes when executing a program. It consists of two main phases: Fetch and Execute.



The fetch/execute cycle of a processor.

1. Fetch Phase:

Purpose: The primary goal of the fetch phase is to retrieve the next instruction from memory.

Operation: The processor accesses the memory location specified by the program counter (PC), which holds the address of the next instruction. The instruction stored in that memory location is then read and transferred to the processor. The instruction is placed on the data bus and sent to the instruction decoder for interpretation.

2. Execute Phase:

Purpose: The execute phase involves carrying out the actions specified by the decoded instruction.

Operation: The instruction decoder translates the binary opcode into control signals that guide the processor's internal components. The processor performs the specific operation indicated by the opcode, such as arithmetic or logic operations, data transfers, or control flow changes. Results may be stored in registers or memory, and flags may be updated based on the outcome of the operation.

3. Cycle Continuation:

After the execute phase, the processor returns to the fetch phase to retrieve the next instruction. The program counter is typically incremented to point to the memory location of the subsequent instruction. This cyclic process repeats for each instruction in the program, creating a continuous loop of fetch and execute phases.

Memory:

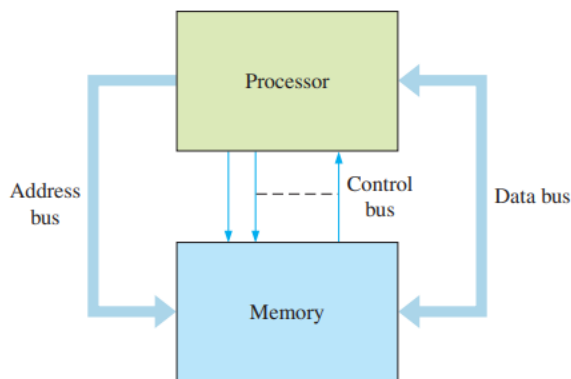
Memory in computer systems is a fundamental component that plays a crucial role in storing and retrieving data for the processor. It facilitates the temporary or permanent storage of instructions and data required for the execution of

programs. The interaction between the processor and memory involves the use of address and data buses, as well as control signals.

Memory Bus:

In computer architecture, a memory bus is a communication pathway that facilitates the transfer of information between the processor and the memory subsystem. It consists of several key components, including the address bus, data bus, and control signals.

1. **Address Bus**: The address bus is unidirectional, meaning that data flows in one direction only. It carries the memory addresses generated by the processor to specify the location in the memory where data is to be read from or written to. The address bits move from the processor to the memory, allowing the processor to select a specific memory location.
2. **Data Bus**: The data bus is bidirectional, allowing data to be transferred in both directions. It carries actual data between the processor and the memory. Data bits can be transferred from the processor to the memory (during write operations) or from the memory to the processor (during read operations).
3. **Control Signals**: Control signals are special signals that manage the operations between the processor and memory. They include commands such as read and write controls, signaling the memory subsystem to perform specific operations. Control signals flow bidirectionally, ensuring that both the processor and memory can communicate effectively.



A processor and memory.

READ OPERATION:

To perform a read operation and transfer data from the memory to the processor, several steps are involved. In the context of an 8-bit data bus and a 16-bit address bus.

Initially, the program counter holds the address of the data to be read from the memory. This address is loaded into the address register and placed onto the address bus. Following this, the program counter is incremented (advanced by one) to the next address and awaits further instructions.

Once the address code is on the bus, the processor's timing and control unit sends a read signal to the memory. At the memory, the address bits are decoded, and the desired memory location is selected. The read signal prompts the contents of the selected address to be placed on the data bus.

Subsequently, the data is loaded into the data register, making it accessible for use by the processor. This sequence of steps completes the read operation. In the provided illustration, each memory location contains one byte of data.

It's important to note that when a byte is read from memory, it is not destroyed; instead, it remains in the memory. This process of "copying" the contents of a memory location without destroying them is termed a nondestructive read. This characteristic ensures that the original data in the memory is preserved after the read operation.

WRITE OPERATION

The write operation is a process in computer architecture that involves transferring data from the processor to the memory. The purpose of this operation is to update or store new information in a specific memory location. The following steps outline the write operation in detail:

Data Transfer from Processor to Memory:

- The process begins with a data byte held in the data register of the processor.
- This data byte is placed onto the data bus, which serves as a communication pathway for data transfer between the processor and memory.

Initiation of Write Signal:

- The processor sends a write signal to the memory.
- The write signal indicates to the memory that it should expect incoming data for storage.

Storage at Memory Location:

- The byte present on the data bus is stored at the memory location specified by the address code.
- The address code determines the precise location in the memory where the data should be written.

Replacing Existing Contents:

- The existing contents of the memory location, as identified by the address code, are replaced by the new data being transferred.
- This means that the data stored at the specified memory location is updated or overwritten with the new information.

Completion of Write Operation:

- The entire process described above constitutes the write operation.
- Once the new data has been successfully written to the specified memory location, the write operation is considered complete.

Addressing Modes:

A processor must address the memory to obtain data or store data. There are several ways in which the processor can generate an address when it is executing an instruction. These ways are called addressing modes and they provide for wide programming flexibility.

There are 5 addressing Modes which are as under:

Immediate Addressing:

Immediate addressing is employed in two-byte instructions, with the first byte serving as the op-code and the second byte as the operand. Examples, such as the load accumulator (LDA) and add to accumulator (ADDA) instructions, utilize immediate addressing.

Inherit Addressing:

Inherent addressing, also known as implied addressing, is a mode in computer architecture where one-byte instructions typically require no explicit operand. The operand is implied by the opcode, which is a mnemonic representation of the instruction. Inherent addressing makes these instructions concise and efficient.

Direct Addressing:

Direct addressing is a mode in computer architecture where the instruction comprises an opcode followed by the actual address of the operand, rather than the operand itself. In direct addressing, the instruction specifies the location in memory where the operand can be found.

Indexed Addressing:

Indexed addressing is associated with the index register. An instruction using indexed addressing consists of the op-code and the offset address. During execution, the offset address is added to the contents of the index register to generate the operand address.

Relative Addressing:

Relative addressing is utilized by branch instructions, enabling the CPU to navigate backward or forward by a specified number of addresses in a program, rather than following the sequential order. In a branch instruction, the first byte is the op-code, and the second byte is the relative address.

I/O Polling:

Polling is a technique used to handle unpredictable events by having the CPU repetitively check or poll peripheral devices that might need attention. When the CPU polls a device, it has to temporarily pause its ongoing task, go through the polling steps, provide help if needed, and then resume from where it left off. However, this method is not very efficient and is only practical for devices that need regular and predictable attention.

Interrupts:

Interrupts are signals or events that temporarily alter the sequence of instructions executed by a computer's central processing unit (CPU). They provide a mechanism for the CPU to promptly respond to external events or conditions, allowing for efficient multitasking, handling asynchronous events, and ensuring timely processing of critical tasks.

Types of Interrupts: -

1. Hardware Interrupts: Triggered by external devices to signal events requiring the CPU's attention. Example: Keyboard input, I/O completion signals

Purpose: Notifies the CPU of events requiring attention.

2. Software Interrupts (Traps): Invoked by software instructions to request services or initiate specific operations.

Example: System calls, specific software operations.

Purpose: Enables the CPU to switch to a specific routine or perform predefined tasks.

3. Exception Interrupts: Generated by the CPU in response to exceptional conditions, errors, or unforeseen events.

Example: Divide-by-zero errors, page faults.

Purpose: Handles and manages unexpected events within the CPU's execution.

DMA (Direct Memory Access):

The DMA (Direct Memory Access) controller is a significant class of bus master devices designed for efficiently transferring large volumes of data between system devices. Unlike the system processor, a DMA controller can accomplish data transfers in a fraction of the time.

To use a DMA controller, the processor initializes the starting source address, starting destination address, and the number of bytes to transfer by writing to the DMA controller's registers.

Computer systems typically employ DMA controllers for tasks such as transferring data between memory and hardware peripherals, like loading a program or data file from a hard drive to memory.

Additionally, DMA controllers facilitate data movement between memory devices, such as transferring data from main memory to cache memory. DMA significantly speeds up data transfers between RAM and specific peripheral devices by bypassing the CPU for certain types of transfers, eliminating the time-consuming fetch and execute cycles required for each CPU read or write operation.

DMA is particularly advantageous for transfers between the disk drive and RAM due to the substantial data volume and the serial nature of these transfers. Generally, a DMA controller can handle data transfers several times faster than the CPU.

Introduction to Types of CPU Instructions:

CPU instructions are the fundamental operations that a processor can execute to perform tasks and manipulate data.

These instructions are the building blocks of machine code, which is the low-level language understood by the CPU.

Understanding the types of CPU instructions provides insights into how a processor processes information and carries out tasks. Here's an introduction to the main types of CPU instructions:

1. Data Transfer Instructions: These instructions move data between registers, memory, and other peripherals. Examples: MOV (Move), LOAD, STORE.
2. Arithmetic Instructions: Perform mathematical operations on data. Examples: ADD, SUB (Subtract), MUL (Multiply), DIV (Divide).
3. Logical Instructions: Execute logical operations, such as AND, OR, and NOT, on binary data. Examples: AND, OR, XOR (Exclusive OR), NOT.
4. Control Transfer Instructions: Direct the flow of program execution. Examples: JMP (Jump), CALL, RET (Return), JZ (Jump if Zero), JNZ (Jump if Not Zero).
5. Conditional Instructions: Execute an operation based on a specified condition. Examples: MP (Compare), JG (Jump if Greater), JL (Jump if Less), JE (Jump if Equal).
6. Input/Output (I/O) Instructions: Facilitate communication between the CPU and external devices. Examples: IN (Input), OUT (Output).
7. System Instructions: Manage system-level operations. Examples: HLT (Halt), INT (Interrupt), CLI (Clear Interrupt), STI (Set Interrupt).